

Walter HTTP API Reference

Official project reference

Version 1.0 | 16 August 2026

1. Overview

Walter exposes a JSON API under `/api/v1`. It is intended for trusted administrative integrations, including the bundled Discord bot. Unless stated otherwise, requests and responses use `application/json`, integer path identifiers are database IDs, and timestamps are ISO 8601 strings or `null`.

2. Authentication and authorization

Create an API key from **Settings** → **API keys** using an account with `admin.api_keys`. The token is displayed once and normally starts with `wlt_`. Send it using either:

```
Authorization: Bearer wlt_your_token
```

or:

```
X-API-Key: wlt_your_token
```

Every documented API endpoint requires `tournaments.manage` or `users.manage`, as specified below. Missing/invalid/revoked credentials return **401**; an authenticated user without permission receives **403**. Revoke keys at `/settings/api-keys/<id>/revoke`. Use HTTPS outside localhost and never place keys in URLs.

3. Conventions

Success bodies are JSON objects. List endpoints wrap records (`{"users": [...]}`). Create operations return **201**; most reads/updates return **200**. Delete returns `{"deleted": true}`. Errors return a JSON error with a relevant **400**, **401**, **403**, **404**, **405**, or **409** status. There is no documented pagination or rate-limit header; list clients should tolerate large arrays. Unknown JSON fields are generally ignored.

Common schemas:

- **User:** `id`, `name`, `email`, `role`, `is_admin`.
- **Tournament:** `id`, `name`, `format`, `structure`, `start_time`, `league_id`, `venue_id`.
- **League:** `id`, `name`, `start_date`, `end_date`, `is_cube_league`, `scoring_system`, `scoring_percentage`, `glicko_enabled`.
- **Round:** `id`, `number`, `matches`.
- **Match:** `id`, `table_number`, `players`, `is_bye`, `completed`. Each player includes `tournament_player_id`, `user_id`, `name`, `dropped`, and sometimes Glicko fields.

4. Users — permission `users.manage`

GET `/api/v1/users`

Lists all users alphabetically.

200 response: `{"users": [User, ...]}`.

POST `/api/v1/users`

Creates a user.

JSON fields: `name` (string, required), `email` (string, optional), `password` (string, optional). Email is trimmed/lowercased.

201 response: `User`.

400: missing `name`.

GET `/api/v1/users/<user_id>`

Returns one `User`. **404** if absent.

PATCH `/api/v1/users/<user_id>`

Updates provided `name`, `email`, and/or non-empty `password`. Returns the updated `User`.

DELETE `/api/v1/users/<user_id>`

Deletes the user and returns `{"deleted": true}`. Referential constraints may make deletion inappropriate for users with event history.

5. Tournaments — `permission tournaments.manage`

GET `/api/v1/tournaments`

Lists tournaments newest first. **200:** `{"tournaments": [Tournament, ...]}`.

POST `/api/v1/tournaments`

Creates a tournament. Fields: `name` (required), `format` (default `Commander`), and `structure` (default `swiss`). **201:** `Tournament`.

```
curl -sS https://walter.example/api/v1/tournaments \
  -H 'Authorization: Bearer wlt_REDACTED' \
  -H 'Content-Type: application/json' \
  -d '{"name": "Friday Draft", "format": "Draft", "structure": "swiss"}'
```

GET `/api/v1/tournaments/<tournament_id>`

Returns `Tournament`; **404** if absent.

PATCH `/api/v1/tournaments/<tournament_id>`

Updates any non-empty `name`, `format`, and/or `structure`; returns `Tournament`.

DELETE `/api/v1/tournaments/<tournament_id>`

Deletes the tournament and returns `{"deleted": true}`.

GET `/api/v1/tournaments/<tournament_id>/standings`

Returns `tournament` and `standings`. Each `standings` row contains `rank`, `tournament_player_id`, `user_id`, `name`, `points`, `omw`, `gw`, `ogw`, and `dropped`.

GET `/api/v1/tournaments/<tournament_id>/rounds`

Returns `tournament` and every `Round`, ordered by round number.

GET `/api/v1/tournaments/<tournament_id>/rounds/latest`

Returns `tournament` and the newest `round`. **404** if the tournament or any round is absent. League-enabled responses may add `glicko_rating` and `glicko_deviation` to players.

6. Leagues — `permission tournaments.manage`

GET `/api/v1/leagues`

Lists leagues alphabetically: `{"leagues": [League, ...]}`.

POST `/api/v1/leagues`

Fields: `name` (required) and `is_cube_league` (boolean, default false). **201**: League.

GET `/api/v1/leagues/<league_id>`

Returns one League; **404** if absent.

PATCH `/api/v1/leagues/<league_id>`

Updates a non-empty `name` and/or boolean `is_cube_league`; returns League.

DELETE `/api/v1/leagues/<league_id>`

Deletes the league and returns `{"deleted": true}`.

GET `/api/v1/leagues/<league_id>/standings`

Returns league and ranked standings. Rows include `rank`, `user_id`, `name`, `played`, `counted_count`, `league_points`, `raw_points`, `wins`, `draws`, `losses`, `colley_rating`, `glicko_rating`, and `glicko_deviation`. Some rating values may be null depending on configuration.

7. Cube leagues and Discord poll transport

All endpoints in this section require `tournaments.manage`.

GET `/api/v1/leagues/<league_id>/play-dates`

Cube leagues only. Returns `league` and `play_dates`, each containing `id`, ISO `play_date`, `is_active`, and `available_cube_count`. **404** when the league is absent or not a cube league.

GET `/api/v1/leagues/<league_id>/cube-votes/<play_date_id>`

Returns the current ballot: `league`, `play_date`, and `cubes`. Each cube has `id`, `title`, `cube_cobra_url`, `image_url`, and aggregate votes. **404** for a mismatched league/date.

POST `/api/v1/discord/cube-polls`

Registers or updates the Discord message mirroring a cube ballot.

Required fields: `league_id`, `play_date_id`, `channel_id`, `message_id`. Discord IDs should be sent as strings.

200: `registered: true`, a poll object, and `cube_vote` ballot.

GET `/api/v1/discord/cube-polls/<message_id>`

Fetches registered poll metadata and the fresh ballot totals. **404** when unregistered.

POST /api/v1/discord/cube-vote

Mirrors a Discord reaction into a Walter vote.

Required fields: `discord_user_id`, `league_id`, `play_date_id`, `cube_id`; `selected` is boolean. The Discord identity must already be connected. The cube must appear on the ballot. A user may select at most three cubes for a play date.

200: refreshed ballot.

403: Discord account not connected.

404: ballot/cube mismatch.

409: selection would exceed three total votes.

8. Discord identity and result reporting

POST /api/v1/discord/authorize (alias: POST /connect)

Permission: `tournaments.manage`. Connects a Discord account to a Walter account using the one-time pass from Settings.

Required: `discord_user_id`, `discord_username`, `one_time_pass`. Optional: `discord_display_name`, `discord_global_name`. JSON and form values are accepted. Leading @ is normalized. The configured Walter Discord username must match case-insensitively.

200: `{"authorized": true, "user": User}`.

400: required field missing.

403: invalid/expired pass, missing Walter username, or mismatch.

409: Discord account belongs to another Walter user.

The pass is consumed after success.

POST /api/v1/discord/report-pairing

Permission: `tournaments.manage`. Reports the connected user's pairing in the latest round.

Required: `discord_user_id`, `tournament_id`, `table_number`. Result fields: `player1_wins` (default 0), `player2_wins` (default 0), `draws` (default 0); all must be non-negative integers.

200: `reported: true`, updated match, and round.

400: invalid score or Commander event (not supported here).

403: identity unconnected or caller not in the pairing.

404: tournament/latest round/table absent.

409: a later round already exists.

9. Worked examples

List standings:

```
curl -sS https://walter.example/api/v1/tournaments/42/standings \
-H 'X-API-Key: wlt_REDACTED'
```

Update a league:

```
curl -sS -X PATCH https://walter.example/api/v1/leagues/7 \  
-H 'Authorization: Bearer wlt_REDACTED' \  
-H 'Content-Type: application/json' \  
-d '{"name":"2026 Cube League","is_cube_league":true}'
```

A robust client should set connect/read timeouts, inspect HTTP status before decoding expected fields, avoid automatic retries for non-idempotent POSTs, and log request IDs locally without logging credentials.

10. Audit and security notes

Walter stores API request metadata and response bodies in the API audit log; authorization headers are redacted. Treat request bodies as auditable operational data. Use a distinct key per integration, a least-privileged owning account, TLS, secret storage, and prompt revocation on suspected exposure. API keys do not replace end-user Discord connection: bot write actions validate the connected Discord identity separately.